

Run the 7-step process on a mini scenario — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `seven_step_practice.md`: the 7 steps only become a habit once you've run them on a real, messy request — this one small scenario is that first run, done on paper, before any project depends on it. **If not:** the first time you tried the process for real would be at the start of Project 1, with a whole project's worth of decisions riding on it.

Write your own answer in `practice/seven_step_practice.md` first, then compare. Read both depths — they're not "wrong, right," they're 2 real levels of the same plan, with real tradeoffs between them.

Scenario: *"Build something that reads a team's daily emails and flags the urgent ones."*

Basic Version

Approach 1 — the direct way

1. **Problem:** the team gets too many emails to read carefully every day, and important ones sometimes get missed. We need something that reads them and points out the urgent ones.
2. **Requirements:** must read new emails, must decide urgent or not, must show the urgent ones somewhere. Must not delete or change the original emails.
3. **Architecture:** one agent — it reads an email and decides if it's urgent.
4. **Components:** an email-fetcher, an urgency-checker, a flagged list.
5. **Data Flow:** email comes in → urgency-checker looks at it → if urgent, add it to the flagged list.
6. **Implementation Plan:** build the urgency-checker first, test it on some example emails, then connect it to real email fetching.
7. **Coding Tasks:** write the checker function, test it, connect a real email source, connect a place to show flagged emails.

This is a reasonable first pass — it covers all 7 steps and would work for a simple version. It's missing detail on ambiguity and failure handling, which is fine for a first attempt.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a buildable plan: real names, real shapes, riskiest part first

Story: a plan that says "an urgency-checker" can't be built without more meetings; one that says `judge_urgency(email) -> (is_urgent, reason)` can. Naming real functions and data shapes turns the plan into work you can start today. **If not:** every vague step would turn into a design argument halfway through coding.

1. **Problem:** a team's shared inbox receives more email than anyone reads carefully; genuinely urgent messages (a client escalation, a deadline today) sometimes sit unread alongside routine ones. We need a system that reviews new emails once a day and surfaces the ones that need immediate attention, without anyone having to read every message.
2. **Requirements:** - Must: read every new email since the last run, judge each one's urgency, write the urgent ones to a place the team actually checks, include *why* each one was flagged. - Must never: modify or delete the original email, silently drop an email without judging it, flag something as urgent with no visible reason.
3. **Architecture:** a single agent, not multiple — this is a classification task (urgent or not) applied to each email independently, with no multi-step reasoning or handoff needed between different specialists. Tying this to Doc11: multiple agents earn their cost when a task genuinely needs different specialized skills working together; a single well-prompted classifier does this job.
4. **Components:** `fetch_new_emails()` (reads from the inbox since last run), `judge_urgency(email) -> (is_urgent, reason)` (the actual agent call), `write_flagged(email, reason)` (writes to wherever the team checks — a shared doc, a Slack message, a labeled inbox folder).
5. **Data Flow:** `fetch_new_emails()` returns a list → each email passed one at a time to `judge_urgency()` → for each `True` result, `write_flagged()` records the email and the reason → nothing happens for `False` results.
6. **Implementation Plan:** first, get `judge_urgency()` working and tested against 5-10 hand-picked example emails (some obviously urgent, some obviously not, a couple of genuinely ambiguous ones) — this is the actual hard part, and it should work correctly before anything touches a real inbox. Then wire up `fetch_new_emails()` against the real email source. Then wire up `write_flagged()`.

7. **Coding Tasks:** (a) write `judge_urgency()` with a clear prompt and a Pydantic-shaped return (`is_urgent: bool`, `reason: str`), (b) write and run a test file with 8-10 example emails and manually check the results, (c) write `fetch_new_emails()` against the real email API, (d) write `write_flagged()`, (e) wire all three together and run once end-to-end on real (but low-stakes) data before trusting it daily.

Approach 2 — the same plan, plus ambiguity, failure points, and a scope line

Story: a one-sentence request is always missing decisions, and each missing decision quietly becomes an assumption in your design. Writing them down — along with what can fail and what v1 won't do — is the part a senior engineer checks first. **If not:** a hidden assumption ("one shared inbox") could be wrong, and you'd find out after building the whole thing around it.

Everything from Approach 1, plus three additions:

Ambiguity surfaced before designing (this is the actual senior skill being tested):

- "Daily emails" — assumed to mean "run once a day, judge everything received since the last run." A genuinely urgent email arriving right after that day's run would sit unflagged for up to 24 hours — worth explicitly flagging to whoever requested this, since "daily" might actually mean "check every few hours" once they picture a real deadline slipping through that window.
- "The team's emails" — assumed to mean one shared inbox. If it actually means "everyone's individual inbox," the architecture changes (per-person credentials, per-person output) — this single assumption changes the whole design, so it's the first thing to confirm before building anything.
- "Urgent" has no given definition — assumed the judging agent infers it from content (mentions of a deadline, an angry tone, a VIP sender) rather than a fixed keyword list, since a fixed list misses anything unanticipated. Flagged as an assumption, not a certainty.

Failure points named, with an explicit response:

- Email source unreachable (API down, credentials expired) → `fetch_new_emails()` should raise a specific error, logged clearly (Doc01's `MissingConfigError` pattern, applied here), not silently return an empty list that looks like "no new emails" to whoever's watching.
- The urgency judgment is genuinely unclear for a specific email → rather than forcing a binary True/False, `judge_urgency()` returns a third option — `needs_human_review` — for cases below a confidence threshold, instead of guessing (this is Doc07's "when NOT to use an agent to force a decision" idea, applied at the level of one classification instead of a whole task).
- Two emails about the same topic arrive close together → out of scope for v1, explicitly. Grouping related emails is a real feature, but it adds real complexity (matching emails by topic) that isn't needed to solve the actual stated problem yet.

Explicit v1 scope line: *"v1 flags individual emails only, does not group related threads, does not distinguish 'urgent to me' from 'urgent to the team,' and runs once daily rather than continuously. All*

three are real, reasonable follow-up features — deliberately left out of v1 so the first version ships and proves the core idea works before adding complexity."

Difference from Basic: Approach 1 names real function signatures and a real data shape, so the plan is buildable on its own, and it tests the hardest, riskiest part (the urgency judgment) before anything else. Approach 2 keeps all of that and treats the one-sentence request as intentionally underspecified — the way real requests actually are — surfaces every place that ambiguity would silently become a wrong assumption, names concrete failure points with a real response for each, and draws an explicit, defensible scope line instead of quietly deciding what's in and out of v1 without saying so.

Which one should you actually write, in a real design conversation? Approach 1's level of detail is the right *default* for a first pass — it's buildable and doesn't over-engineer a simple request. But Approach 2's 3 additions (surfaced ambiguity, named failure points, an explicit scope line) are what turn a plan into one a senior engineer would actually sign off on without follow-up questions — do all three, every time, even for a request this small. It takes a few extra minutes and it's exactly what Doc16's exercises test you on at a bigger scale.